Overview and Plan

FraudFlow — 21-Hour Execution Plan

Why this idea wins

Looking at the judging criteria (Innovation, Technical Complexity, Product Experience, Impact), FraudFlow hits all four harder than most teams will. More importantly, it's clearly targeting **4 prize categories simultaneously**: TD Fraud, Bitdeer Production-Ready, Moorcheh Memory, and Google Community Impact. That's your "floor" of wins — any one of them is likely.

Suggested Changes & Improvements

Before the plan, a few strategic tweaks to the original idea:

1. Drop LangGraph, use a simpler agentic loop. LangGraph adds setup overhead. A clean Python state machine with Claude/GPT-4o calls at decision nodes achieves the same demo effect in half the time. If you have LangGraph experience, keep it — otherwise cut it.

2. Hardcode your 3 demo scenarios as "playbooks." Don't try to make fraud detection generalize from real data. Build 3 crisp, scripted scenarios (rogue app, transaction anomaly, social engineering) with pre-seeded fake data. Judges respond to a compelling demo, not a robust model.

3. Replace Isolation Forest with a rule-based + LLM hybrid. A pure ML model on fake data will look weak. Instead, use lightweight heuristics (rate limits, time-of-day, permission scope mismatch) to generate risk signals, then pass them to GPT-4o/Claude to produce a human-readable verdict. This looks more impressive and is easier to debug under pressure.

4. Add a "Trust Score" badge to the consumer UI. A simple visual (green/yellow/red score per connected app) is what judges will remember and photograph. Make it the centerpiece of your consumer dashboard.

5. Angle for IBM prize too. You could route your LLM calls through IBM watsonx.ai instead of OpenAI with minimal effort — that unlocks a 5th prize category.

Team Split (assuming 3 people)

Person	Role
A	FastAPI backend — mock Open Banking gateway + behavior logging
B	Agentic decision engine — LLM integration + Benford's Law + risk scoring logic + Moorcheh
C	React frontend — consumer dashboard + analyst view

Hour-by-Hour Timeline

Hours 1–2: Foundation

- Repo setup, agree on data models (App, APICall, RiskEvent, User)
- Seed 3 fake fintech apps with synthetic API call logs
- Person A starts FastAPI skeleton; Person C scaffolds React app

Hours 3–6: Core Backend

- Mock Open Banking endpoints: `GET /accounts`, `GET /transactions`, `POST /payments`
- Middleware that intercepts every call and logs: app_id, endpoint, timestamp, data_volume, frequency
- Heuristic rule engine: flag if call rate > threshold, if time is 2–4am, if endpoint doesn't match app category

Hours 7–10: AI Layer

- LLM prompt that takes risk signals and outputs: verdict (APPROVE / FLAG / BLOCK), confidence score, plain-English explanation
- Moorcheh integration: store app behavioral fingerprints, query "has this pattern been seen before?"
- Wire the three components together: API call → rules → Moorcheh query → LLM → decision

Hours 11–14: Frontend

- Consumer view: connected apps list, trust scores, recent alerts ("FraudFlow blocked an unusual data pull at 3:12am")
- Analyst view: live feed of API calls, color-coded by risk level, drill-down panel for flagged events
- Trigger demo scenarios from the UI with a button

Hours 15–17: Demo Polish

- Pre-load all 3 scenarios so they run cleanly on button click
- Make sure each scenario ends with a visible, satisfying outcome (block notification, trust score dropping, alert firing)
- Error handling so nothing crashes during the live demo

Hours 18–19: README + Devpost writeup

- Architecture diagram (draw it, photograph it, put it in the README)
- Write the Devpost submission hitting each prize category's language explicitly
- Record a 2-minute demo video as backup

Hours 20–21: Buffer / Sleep

- Fix whatever broke
- Practice the 3-minute pitch out loud at least twice

Tech Stack (Recommended)

- **Backend:** Python + FastAPI
- **AI:** Claude API (via Anthropic) or GPT-4o — whichever your team knows
- **Memory:** Moorcheh boilerplate (they provide a Python starter kit — grab it at their workshop)
- **Frontend:** React + Tailwind + shadcn/ui for the dashboard
- **Data:** JSON flat files or SQLite — don't waste time on a real database
- **Hosting:** Run locally for the demo; push to GitHub

The Pitch Frame (memorize this)

> "Canada's Open Banking launch in 2026 will let third-party apps connect directly to your bank account. That's powerful — and it's a new fraud surface no one has solved yet. FraudFlow sits as a middleware layer between the bank and the fintech ecosystem, watching how apps behave, not just how users behave. When an app acts like a data harvester instead of a budgeting tool, we catch it before your money moves."

That one paragraph hits Innovation, Impact, and Technical Complexity in 20 seconds.

Most Critical Thing

Your 3 demo scenarios must be airtight. Judges at hackathons remember what they *saw*, not what they *heard*. If Scenario 3 (the senior/scammer one) fires cleanly — app registered 48 hours ago, excessive permissions, FraudFlow blocks it and sends an alert — that's a winning moment. Build toward that moment first.

Good luck — this is a genuinely strong idea. What's your team size and what's everyone comfortable building?

Detailed CC Plan

The key to parallel work with Claude Code is a **very detailed `CLAUDE.md` file** — this is Claude Code's bible for your repo. Before any of you write a single line of code, you need this file to exist so every Claude Code instance across all 3 laptops has the same understanding of the architecture, contracts, and conventions.

Here's exactly what to do:

Step 1: Create the Project Structure First

Have one person run this to scaffold the repo skeleton:

```
fraudflow/
├── CLAUDE.md              ← the most important file
├── README.md
├── .env.example
├── backend/
│   ├── main.py
│   ├── requirements.txt
│   └── app/
│       ├── gateway/        ← Person A (Open Banking API)
│       ├── profiler/       ← Person B (behavior + Benford)
│       ├── memory/         ← Person B (Moorcheh)
│       ├── agent/          ← Person B (LLM decision engine)
│       └── models.py       ← shared data models (do this FIRST)
├── frontend/
│   ├── package.json
│   └── src/
│       ├── components/
│       └── views/
│           ├── ConsumerDashboard/   ← Person C
│           └── AnalystDashboard/    ← Person C
└── data/
    └── seeds/              ← fake app + transaction data
```

Push this skeleton to GitHub before anyone else clones.

Step 2: Write the CLAUDE.md

This is what you actually prompt Claude Code with. Here's a template tailored to FraudFlow — paste this in and customize the names:

FraudFlow — CLAUDE.md

Project Overview
FraudFlow is an AI-powered fraud detection middleware layer for Canada's Open Banking ecosystem. It sits between a simulated TD Open Banking API gateway and third-party fintech apps, detecting anomalous app behavior using heuristics, Benford's Law analysis, Moorcheh memory, and LLM reasoning.

Architecture
- **Backend**: Python 3.11 + FastAPI (port 8000)
- **Frontend**: React 18 + Tailwind CSS + shadcn/ui (port 5173)
- **Memory**: Moorcheh Python SDK
- **AI**: Anthropic Claude API (claude-sonnet-4-20250514)
- **Data**: SQLite via SQLModel for dev simplicity

Module Ownership (DO NOT modify other modules without coordinating)
- `backend/app/gateway/` — Open Banking API simulation (Person A)
- `backend/app/profiler/` — Behavior profiling + Benford's Law (Person B)
- `backend/app/memory/` — Moorcheh integration (Person B)
- `backend/app/agent/` — LLM decision engine (Person B)
- `frontend/src/` — React dashboard (Person C)

Shared Contracts (DO NOT change these without telling everyone)
All inter-module communication uses the models defined in `backend/app/models.py`. These are the source of truth.

Key models:
- `APICallLog` — every call a third-party app makes to the gateway
- `RiskSignals` — output of the profiler (Benford score, rate flags, etc.)
- `FraudDecision` — output of the agent (verdict, confidence, explanation)
- `AppProfile` — registered third-party app with trust score

API Contracts (backend → frontend)
The frontend consumes these endpoints. Backend must implement exactly these:
- `GET /api/apps` — list all connected apps with trust scores
- `GET /api/apps/{app_id}/calls` — API call history for an app
- `GET /api/alerts` — recent FraudDecision events (FLAG or BLOCK)
- `POST /api/demo/trigger/{scenario}` — trigger demo scenario 1, 2, or 3
- `GET /api/apps/{app_id}/benford` — Benford deviation data for charting

Demo Scenarios (seed data lives in data/seeds/)

- Scenario 1: `rogue_app` — budgeting app making calls at 3am, wrong endpoints
- Scenario 2: `transaction_anomaly` — payment initiation with mismatched user profile
- Scenario 3: `social_engineering` — app registered 48hrs ago, excessive permissions, Benford deviation detected

Environment Variables (see .env.example)
- `ANTHROPIC_API_KEY`
- `MOORCHEH_API_KEY`
- `DATABASE_URL` (defaults to sqlite:///./fraudflow.db)

Code Conventions
- All backend functions must have type hints
- API responses always use the models from models.py, never raw dicts
- Frontend fetches always go through `src/api/client.ts` (single axios instance)
- No hardcoded strings — use constants files

Running Locally
```bash
# Backend
cd backend && pip install -r requirements.txt
uvicorn main:app --reload

# Frontend
cd frontend && npm install && npm run dev
```

What "Done" Looks Like Per Module

- Gateway: all 5 endpoints respond, every call writes an APICallLog to DB
- Profiler: given a list of APICallLogs, returns a RiskSignals object
- Agent: given RiskSignals + app context, returns a FraudDecision
- Frontend: consumer + analyst views render with mock data from /api endpoints

---

Step 3: Initialize models.py Together (30 min, all 3 of you)

Before splitting up, **all 3 sit down and finalize `models.py` together**. This is the handshake file. Once it's committed, nobody changes it without a group decision. Claude Code on every laptop will use it as the contract.

Tell Claude Code:
> "Create `backend/app/models.py` with SQLModel models for APICallLog, RiskSignals, FraudDecision, and AppProfile based on the specs in CLAUDE.md"

Review it together, commit it, then split.

Step 4: What Each Person Tells Claude Code

Once you split, each person opens Claude Code in the repo and starts with:

Person A:
> "I am working on `backend/app/gateway/`. Read CLAUDE.md and models.py first. Build the mock Open Banking FastAPI router with the 5 endpoints listed. Every incoming API call must log an APICallLog to the database. Do not touch any other module."

Person B:
> "I am working on `backend/app/profiler/`, `backend/app/memory/`, and `backend/app/agent/`. Read CLAUDE.md and models.py first. The profiler takes a list of APICallLog objects and returns RiskSignals including a Benford deviation score. The agent takes RiskSignals and returns a FraudDecision using the Anthropic API. Do not touch the gateway or frontend."

Person C:
> "I am working on `frontend/src/`. Read CLAUDE.md first. Build the consumer and analyst dashboard views in React + Tailwind. Use the API endpoints listed in CLAUDE.md — they don't exist yet so mock the responses locally using the shape of the models described. Do not touch the backend."

Step 5: Merging

When you're ready to connect everything, the integration surface is tiny because you defined it upfront — it's just Person C swapping mock data for real API calls, and Person A/B making sure the endpoints match the contract. Should take under an hour if the contracts held.

The single biggest risk to parallel work is someone quietly changing a model or endpoint shape. The CLAUDE.md + models.py combo prevents that. Treat those two files as sacred.

Feature Overview

FRAUDFLOW — Full Feature Breakdown

AI fraud detection middleware for Canada's Open Banking future

---

CORE SYSTEM

Third-party fintech app → simulated Open Banking API gateway → FraudFlow intercepts every request → behavior profiler scores risk → memory layer checks known patterns and past activity → decision engine returns APPROVE / FLAG / BLOCK → consumer and analyst dashboards update in real time

---

MAIN BUILD

1. THIRD-PARTY APP REGISTRY + TRUST PROFILE - Backend - Abdullah

- Every connected fintech app has a profile
- Stores: app name, category, permissions requested, registration date, trust score, status
- Categories can be things like budgeting, payments, lending, tax, investing
- This becomes the foundation for comparing what an app says it is versus how it actually behaves
- Consumer dashboard uses this to show all connected apps
- Analyst dashboard uses this to monitor the ecosystem at the app level

Uses

- FastAPI

- SQLite / SQLModel
- Seeded synthetic fintech app data

Why it matters

- Gives the whole product structure
- Makes the demo feel like a real open banking ecosystem instead of random API logs

---

2. SIMULATED OPEN BANKING API GATEWAY - Backend - Abdullah

- Mock gateway that represents what a bank would expose to third-party apps
- Endpoints can include:
    - `GET /accounts`
    - `GET /transactions`
    - `POST /payments`
    - `GET /balances`
    - `POST /consent`
- Every request from a third-party app passes through this layer first
- FraudFlow sits in front of or beside the gateway and inspects every call before the request is allowed to complete

Uses

- FastAPI routers
- Middleware / request interception

- Auth token simulation
- Permission scope checks

Why it matters

- This is the core product surface
- It makes your idea feel infrastructure-grade, which helps with the Bitdeer angle

---

3. API CALL TELEMETRY + EVENT LOGGING - Backend - Abdullah

- Every incoming request becomes a structured event
- Log fields:
    - app_id
    - app category
    - endpoint accessed
    - timestamp
    - time of day
    - request frequency
    - permission scope used
    - data volume requested
    - user_id
    - whether request was allowed or blocked
- This becomes the raw signal for profiling, memory lookups, and dashboard visualizations

Uses

- Shared `APICallLog` model
- SQLite table for event history
- Optional live polling endpoint for dashboards

Why it matters

- Without this, there is no fraud engine
- It also gives you something visual and real-time for the analyst view

---

4. PERMISSION SCOPE MISMATCH DETECTION - Core Risk Logic

- Checks whether an app is accessing data that does not match its declared purpose
- Example:
    - budgeting app requesting payment initiation
    - tax app requesting full transaction history every few minutes
    - newly registered app asking for excessive permissions immediately
- This is one of the cleanest and most judge-friendly fraud signals

Uses

- Rule-based checks
- App category → allowed behavior mapping
- Scope mismatch flags inside `RiskSignals`

Why it matters

- Easy to explain in one sentence

- Easy to demo
- Feels highly relevant to open banking

---

5. BEHAVIOR PROFILER - Risk Engine

- Builds a behavior fingerprint for each app
- Measures:
  - call frequency
  - endpoint mix
  - time-of-day anomalies
  - burst behavior
  - unusual data pull volume
  - behavior drift from baseline
- Produces a `RiskSignals` object for each event or batch of recent events
- You do **not** need to overcomplicate this for the hackathon
- Best version for a weekend build is rules first, anomaly model second

Uses

- Python scoring functions
- Threshold-based heuristics
- Optional anomaly model later such as Isolation Forest

Why it matters

- This is where FraudFlow becomes more than a dashboard

- Lets you say you are detecting app behavior anomalies, not just fraudulent transactions

---

6. BENFORD / NUMERIC PATTERN CHECKS - Optional Core Signal

- Looks at numerical distributions in payment or transaction request data
- Can help flag suspicious synthetic activity or weird transaction patterns
- Best used as a supporting signal, not the headline feature
- Show it in the analyst dashboard as one risk component rather than the main engine

Uses

- Lightweight statistics
- Risk score contribution
- Visual chart for analyst panel

Why it matters

- Adds technical depth
- Gives the project a more sophisticated look without making the entire system depend on it

---

7. MOORCHEH MEMORY LAYER - Memory / Context

- Stores historical risk events, trusted apps, user baselines, and prior fraud patterns
- When a new event comes in, the system can ask:
    - Has this app behaved like this before?

    - Has a similar app pattern been flagged before?
    - Has this user historically trusted this app?
    - Is this behavior similar to known data-harvesting patterns?
- Lets FraudFlow reason using historical context instead of only current event data

Uses

- Moorcheh boilerplate / SDK
- Semantic retrieval
- Memory records for apps, users, and incidents

Why it matters

- This is your strongest Moorcheh prize hook
- Also makes the decision engine look smarter and more contextual

---

8. AGENTIC DECISION ENGINE - AI Layer

- Takes the structured risk signals and memory results
- Returns:
    - verdict: APPROVE / FLAG / BLOCK
    - confidence
    - short explanation
    - recommended action
- Should sound like a bank fraud analyst, not a generic chatbot
- Example output:

    - “Blocked: budgeting app requested payment initiation outside its declared scope and showed abnormal overnight access bursts inconsistent with peer apps.”

Uses

- Claude or GPT
- Prompt template with strict JSON output
- Small Python orchestration layer

Why it matters

- This is what gives you the AI story
- Also turns raw risk signals into something judges can instantly understand

---

9. ACTION LAYER - Enforcement

- Once the decision is made, FraudFlow can:
    - allow request
    - log only
    - flag for review
    - request user confirmation
    - throttle future calls
    - revoke token
- For the demo, focus on **allow / flag / block**
- User confirmation can be a strong stretch feature if time permits

Uses

- Gateway middleware response handling
- Status updates in DB
- Triggered alert records

Why it matters

- This is the “so what” moment
- It proves FraudFlow is a decision-making layer, not just a monitoring tool

---

10. CONSUMER DASHBOARD - Frontend

- View for regular banking users
- Shows:
    - connected apps
    - trust score for each app
    - permissions granted
    - recent suspicious activity
    - blocked attempts
- Core hero moment:
    - “This app tried to access data outside its expected behavior. FraudFlow blocked it.”

Uses

- React
- Tailwind

- shadcn/ui
- mock cards, trust score badges, alerts feed

Why it matters

- This is your cleanest product story for judges
- Makes the problem feel real to everyday users

---

11. ANALYST DASHBOARD - Frontend

- View for TD or fraud operations teams
- Shows:
    - live stream of third-party API activity
    - suspicious apps ranked by risk
    - heatmap or chart of anomalous behavior
    - event drill-down with explanation and memory hits
- Should feel like a control center

Uses

- React tables/cards/charts
- polling or mock real-time updates
- event detail drawer or modal

Why it matters

- This is your enterprise side

- Helps sell the “bank infrastructure” angle

---

12. DEMO SCENARIO ORCHESTRATOR - Demo Backbone

- Buttons that trigger your three scripted scenarios
- Scenario 1: rogue budgeting app
- Scenario 2: payment anomaly through third-party app
- Scenario 3: social-engineering tax app / data harvester
- Each scenario should create visible cause and effect:
    - app behavior changes
    - risk score spikes
    - memory hit appears
    - agent explains decision
    - dashboard updates
    - request gets blocked or flagged

Uses

- seed scripts
- `/api/demo/trigger/{scenario}`
- deterministic fake data

Why it matters

- This is one of the most important features in the whole project
- It makes the demo reliable

---

13. ALERTS + AUDIT TRAIL - Shared Product Feature

- Stores every important decision in a readable event history
- Includes:
  - what happened
  - why it was risky
  - what action was taken
  - when it happened
- Can be shown in both dashboards
- Makes the system feel compliant and enterprise-ready

Uses

- `FraudDecision` model
- alerts table / endpoint
- event timeline UI

Why it matters

- Great for Bitdeer “production-ready” language
- Adds clarity and polish

RECOMMENDED MVP CUT LINE

If you need the cleanest realistic MVP, these are the must-have features:

1. Third-party app registry

2. Simulated API gateway
3. API call logging
4. Rule-based behavior profiler
5. Moorcheh memory lookup
6. LLM decision engine
7. Consumer dashboard
8. Analyst dashboard
9. Three triggerable demo scenarios

That is enough to tell a complete story.

SUGGESTED TEAM SPLIT

PERSON A - Backend / Gateway

- app registry
- open banking API simulation
- logging
- action layer
- scenario trigger endpoints

PERSON B - AI / Profiler / Memory

- risk signal generation
- Benford support
- Moorcheh integration

- decision engine
- final verdict formatting

PERSON C - Frontend

- consumer dashboard
- analyst dashboard
- alerts feed
- trust score UI
- scenario controls

THE PRODUCT STORY

“FraudFlow is the AI fraud detection layer for Canada’s Open Banking future. Instead of only detecting suspicious user transactions, it detects suspicious third-party app behavior before damage happens. When a fintech app acts outside its declared purpose, abuses permissions, or matches known malicious patterns, FraudFlow flags, explains, and blocks it in real time.”

WHAT TO BUILD FIRST

Build in this order:

1. App registry
2. Gateway + call logging
3. Risk signals
4. Memory lookup
5. LLM verdict

6. Scenario triggers
7. Consumer dashboard
8. Analyst dashboard
9. Extra polish